COS 214 Practical 4

1. System Description and Team Details

1.1 Project Title & Domain Explanation

TaskForge: Hierarchical Work Processing & Package Delivery System (Logistics & Supply Chain Domain).

Domain Architecture & Business Problem

TaskForge models an enterprise fulfillment and supply chain network inspired by global logistics operations (such as DHL, Takealot, or Amazon Supply Chain). The system processes physical package items through multi-level warehouse container hierarchies while managing dynamic shipping upgrades and parcel delivery lifecycles:

- **Hierarchical Packaging (Composite Pattern)**: Warehouses are structured as nested containers (`CargoGroup`). A top-level Distribution Center contains regional Sector Hubs, which contain Shipping Pallets, which contain individual Parcels (`PackageItem`). Total container weights are calculated recursively across all nested levels.
- **Parcel Lifecycle Management (State Pattern)**: Each parcel transitions through operational states (`OrderPlacedState` to `InTransitState` to `DeliveredState` / `FailedDeliveryState`). State objects manage transition logic and enforce protection guards (e.g. rejecting duplicate dispatches on delivered orders).
- **Dynamic Shipping Upgrades (Decorator Pattern)**: Parcels receive optional value-added services at runtime (`ExpressShippingDecorator` adding priority fees and expedited dispatch, `InsuranceDecorator` logging policy coverage limits) wrapped dynamically around base packages without class explosion.
- **Warehouse Auditing & Priority Dispatch (Iterator Pattern)**: Auditing systems traverse live warehouse containers safely (`SnapshotIterator` capturing immutable tree snapshots to protect against mid-audit inventory movements) while dispatch engines filter high-priority express/insured items (`PriorityFilteredIterator`).

1.2 Team Information

- **1**: Shreya Keogh (`u25245962`)
- **2**: Showlon Hunter (`u25502469`)
- **3**: Enock Nyamweya (`u25619871`)

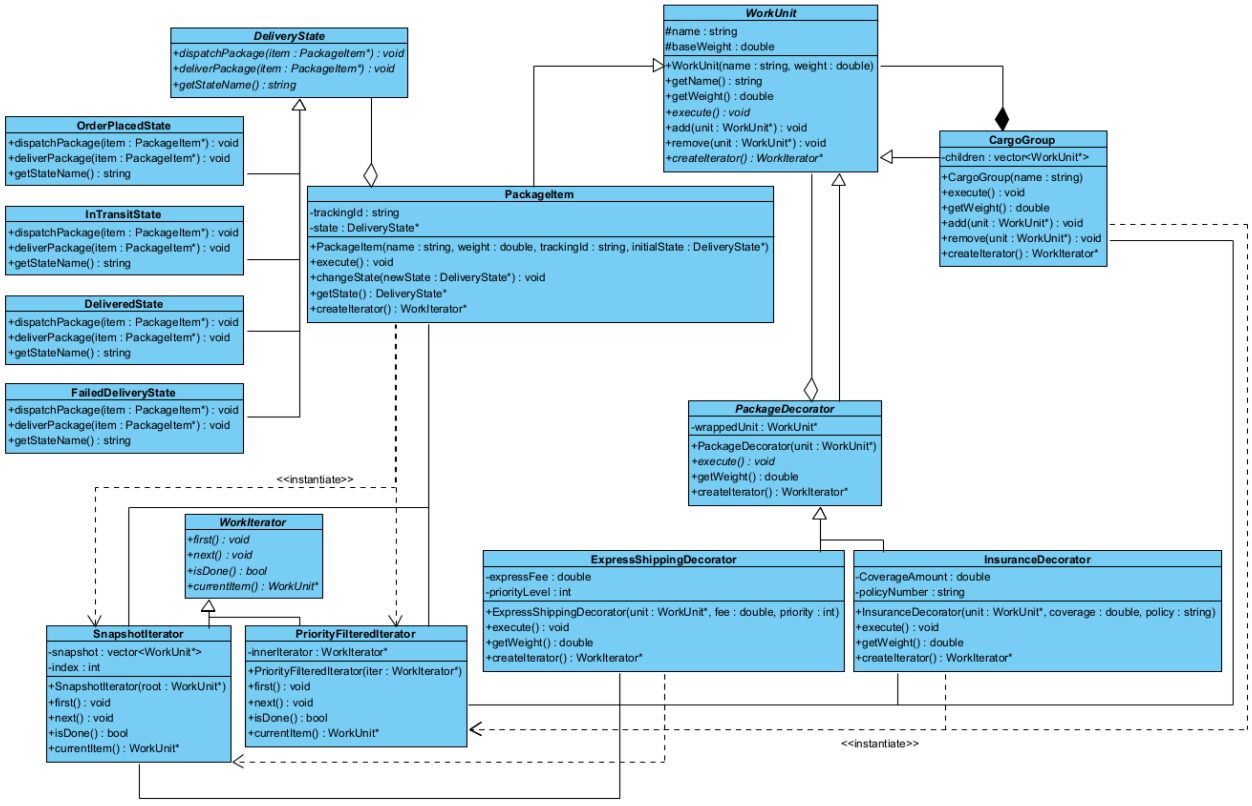
1.3 Repository URL

- **GitHub Repository**: https://github.com/enockNyamweya/PRAC4-COS214

2. UML Class Diagram & GoF Participant Mapping

2.1 UML Class Diagram

The UML Class Diagram illustrates the TaskForge enterprise logistics system architecture, modeling the Composite (`WorkUnit` , `CargoGroup` , `PackageItem`), Decorator (`PackageDecorator` , `ExpressShippingDecorator` , `InsuranceDecorator`), State (`DeliveryState` , `OrderPlacedState` , `InTransitState` , `DeliveredState` , `FailedDeliveryState`), and Iterator (`WorkIterator` , `SnapshotIterator` , `PriorityFilteredIterator`) GoF design patterns.



2.2 GoF Participant Mapping Table

Pattern	GoF Role	System Class Name	Description / Responsibilities
Composite	`Component`	`WorkUnit`	Abstract base class declaring interface for composite and leaf elements (`execute`, `getWeight`, `add`, `remove`, `createIterator`).
Composite	`Composite`	`CargoGroup`	Manages child `WorkUnit*` elements in a `std::vector`, calculating total weight recursively and executing children.
Composite	`Leaf`	`PackageItem`	Represents an individual parcel with tracking ID, weight, and delegates lifecycle behavior to `DeliveryState*`.
State	`Context`	`PackageItem`	Maintains pointer to current `DeliveryState*` and delegates state-dependent operations.
State	`State`	`DeliveryState`	Abstract interface defining polymorphic actions (`dispatchPackageItem`, `deliverPackageItem`, `getStateName`).
State	`ConcreteState`	`OrderPlacedState`	Handles package dispatch; transitions package to `InTransitState`.
State	`ConcreteState`	`InTransitState`	Handles final delivery; transitions package to `DeliveredState`.
State	`ConcreteState`	`DeliveredState`	Enforces post-delivery protection guards rejecting further dispatch/delivery attempts.
State	`ConcreteState`	`FailedDeliveryState`	Handles delivery failures and retries delivery back to `InTransitState`.
Decorator	`Component`	`WorkUnit`	Base interface extended by abstract decorator.

Pattern	GoF Role	System Class Name	Description / Responsibilities
Decorator	`Decorator`	`PackageDecorator`	Abstract decorator wrapping a `WorkUnit*` pointer and forwarding interface calls.
Decorator	`ConcreteDecorator`	`ExpressShippingDecorator`	Dynamically adds priority fee and express handling logs to a wrapped package.
Decorator	`ConcreteDecorator`	`InsuranceDecorator`	Dynamically adds insurance coverage limits and policy logging to a wrapped package.
Iterator	`Aggregate`	`WorkUnit`	Declares factory method `createIterator()` returning `WorkIterator*`.
Iterator	`Iterator`	`WorkIterator`	Abstract interface defining `first()`, `next()`, `isDone()`, `currentItem()`.
Iterator	`ConcreteIterator`	`SnapshotIterator`	Pre-captures composite tree pointers into a snapshot array at initialization to ensure traversal-modification safety.
Iterator	`ConcreteIterator`	`PriorityFilteredIterator`	Wraps an inner `WorkIterator*` and filters elements on the fly to only yield express or insured items.

3. Design & Memory Ownership Rationale

3.1 Inheritance vs. Composition Strategy

- **Part-Whole Hierarchy**: `CargoGroup` uses composition to hold a collection of `WorkUnit*` pointers, allowing nested structures of arbitrary depth (Warehouse → Sector → Pallet → Parcels).
- **Dynamic Decorators**: `PackageDecorator` uses object composition over inheritance to allow dynamic runtime stacking (`InsuranceDecorator` wrapping `ExpressShippingDecorator` wrapping `PackageItem`) without class explosion.
- **State Encapsulation**: `PackageItem` uses aggregation (`DeliveryState* state`) to reference and transition between state objects dynamically at runtime.

3.2 Polymorphic Memory Ownership Policy

To prevent memory leaks and double-free corruption:

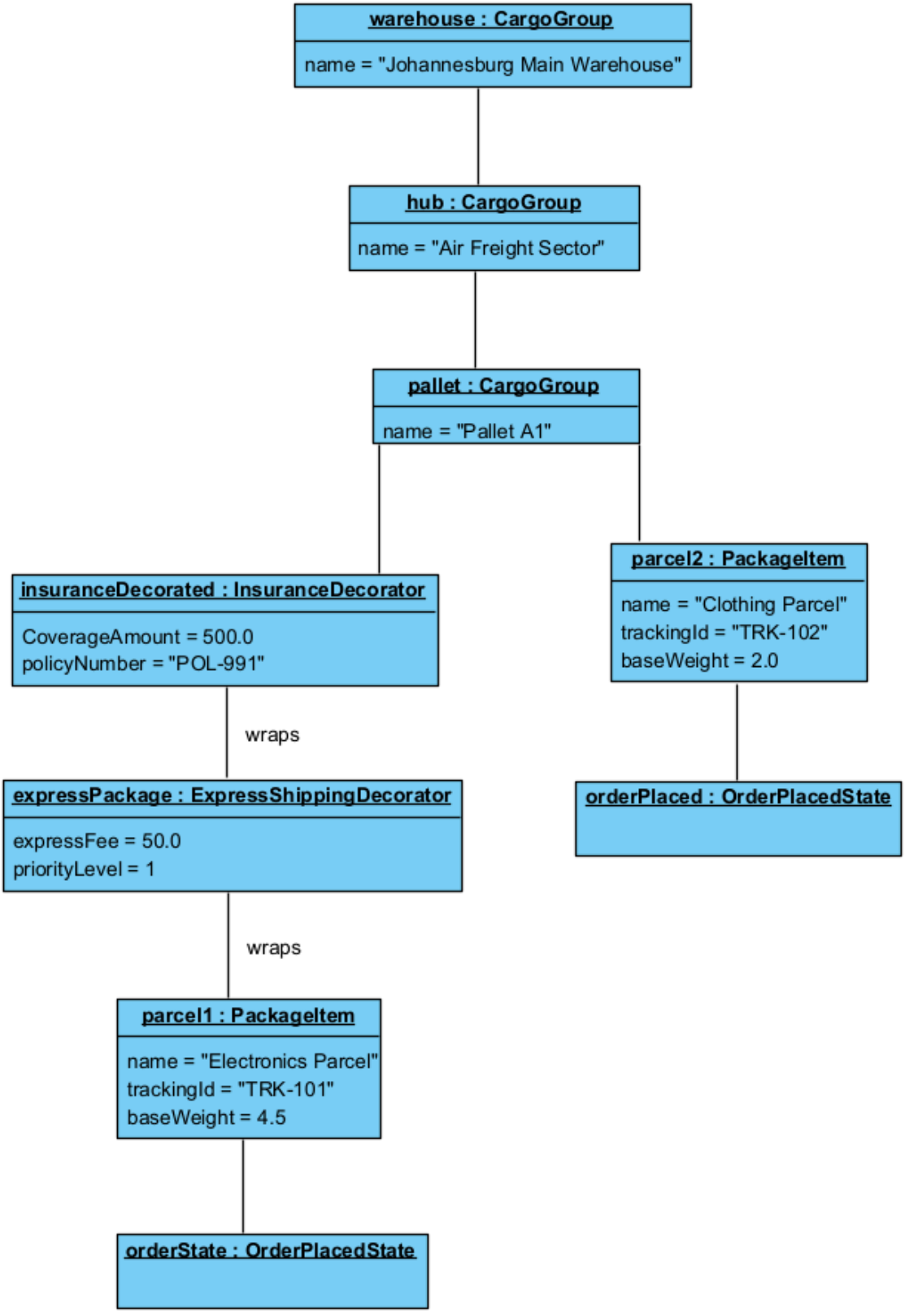
1. **`CargoGroup::~CargoGroup()`** : Recursively deletes all child pointers stored inside `children` vector upon container destruction.
2. **`PackageDecorator::~PackageDecorator()`** : Deletes `wrappedUnit` upon decorator destruction.
3. **`PackageItem::~PackageItem()`** : Deletes `DeliveryState* state` upon parcel destruction.
4. **`SnapshotIterator::~SnapshotIterator()`** : Clears vector references without deleting `WorkUnit*` targets (iterators only reference, never own).
5. **`PriorityFilteredIterator::~PriorityFilteredIterator()`** : Automatically deletes its `inneriterator` pointer.

4. Object Diagram & State Diagram

4.1 UML Object Diagram (3-Level Nesting Hierarchy)

The object diagram illustrates the instantiated runtime object graph featuring a 3-level composite warehouse container hierarchy, stacked decorators, and parcel delivery states:

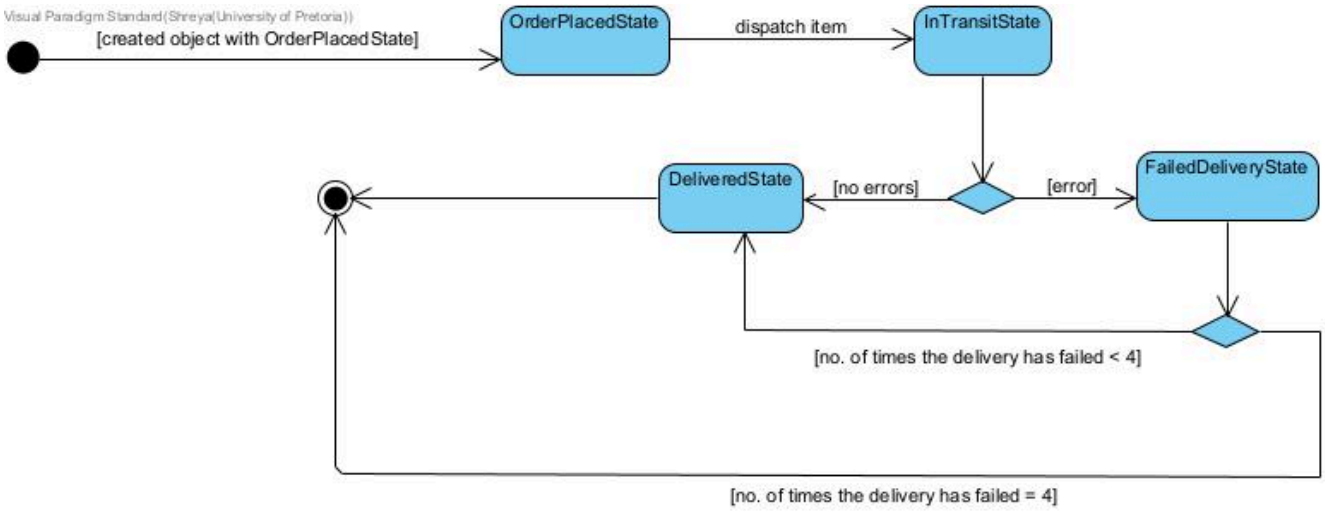
- **Level 0 Root**: `warehouse : CargoGroup` ("Johannesburg Main Warehouse")
- **Level 1 Hub**: `hub : CargoGroup` ("Air Freight Sector")
- **Level 2 Container**: `pallet : CargoGroup` ("Pallet A1")
- **Level 3 Decorator Stack & Leaf**:
- `insuranceDecorated : InsuranceDecorator` wrapping `expressDecorated : ExpressShippingDecorator` wrapping `parcel1 : PackageItem` linked to `orderState : OrderPlacedState`.



4.2 UML State Diagram (Parcel Delivery Lifecycle)

The state diagram illustrates the complete parcel lifecycle state machine and valid operational state transitions (`OrderPlacedState` → `InTransitState` → `DeliveredState` /

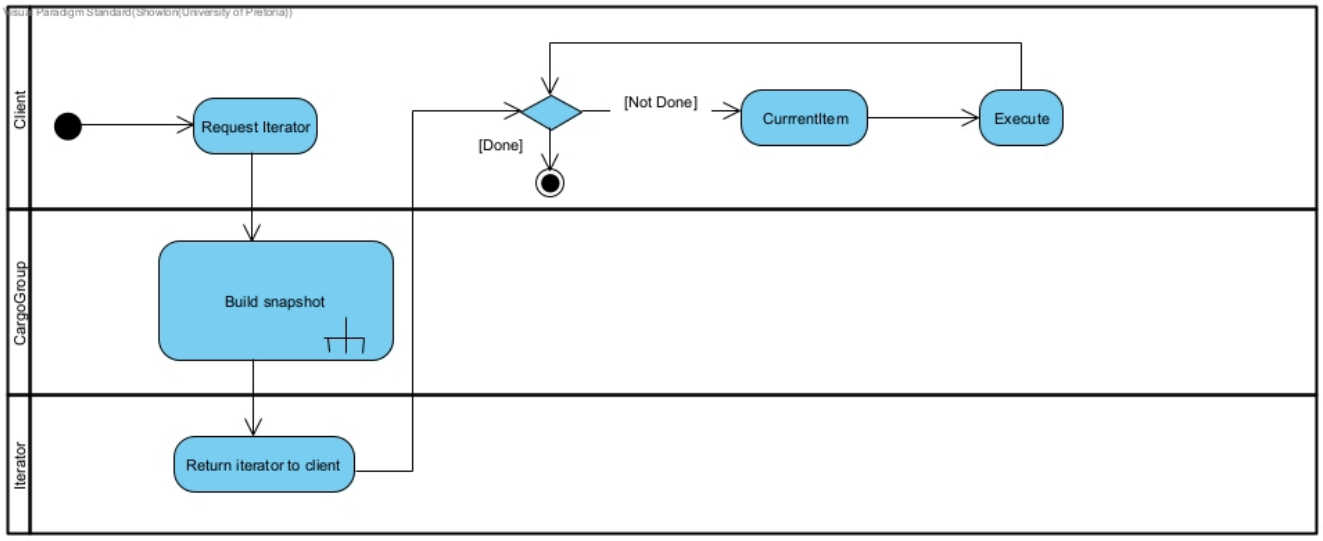
`FailedDeliveryState` → retry back to `InTransitState` / `DeliveredState`) along with protection guards enforcing state invariants.



5. Three UML Activity Diagrams

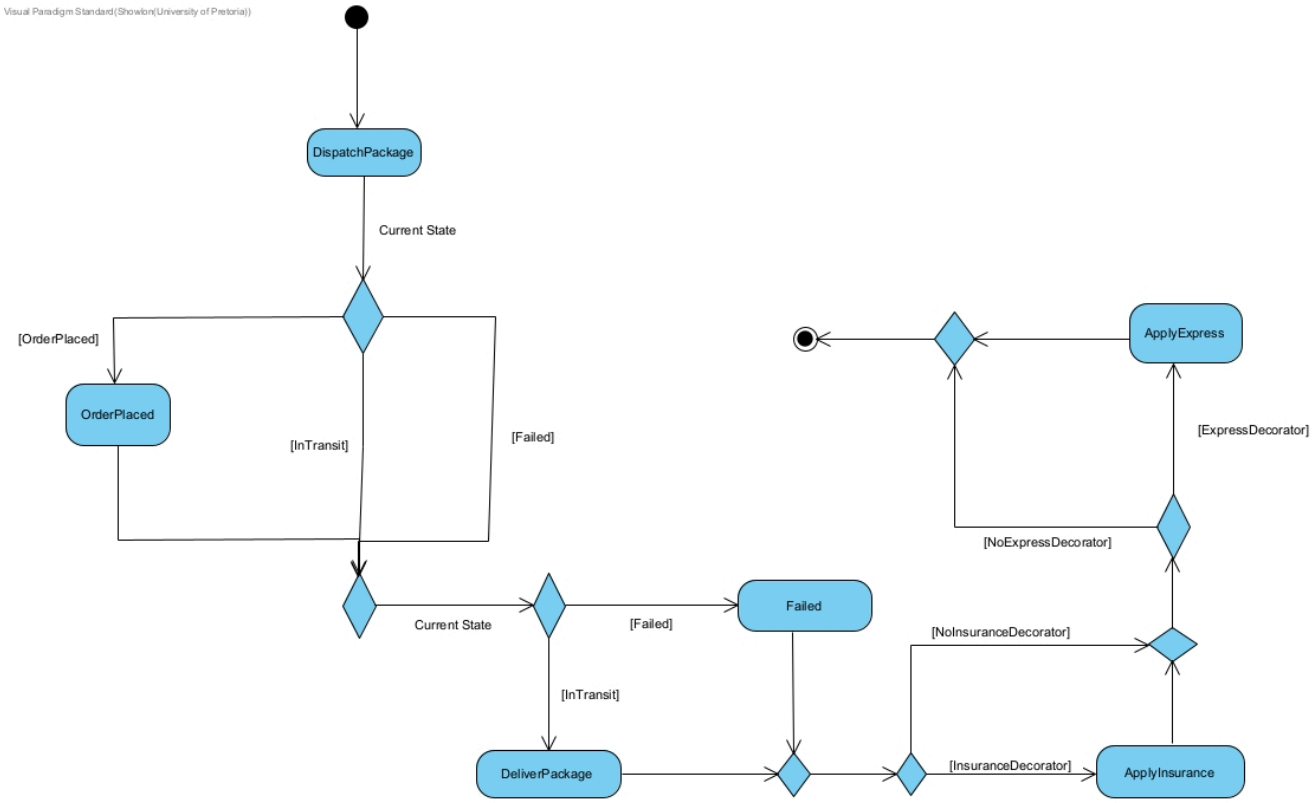
5.1 Activity Diagram 1: Traversal Loop Workflow (`SnapshotIterator`)

Visualizes the initialization, depth-first snapshot creation, `while(!isDone())` iteration loop, and iterator destruction for safe warehouse container auditing.



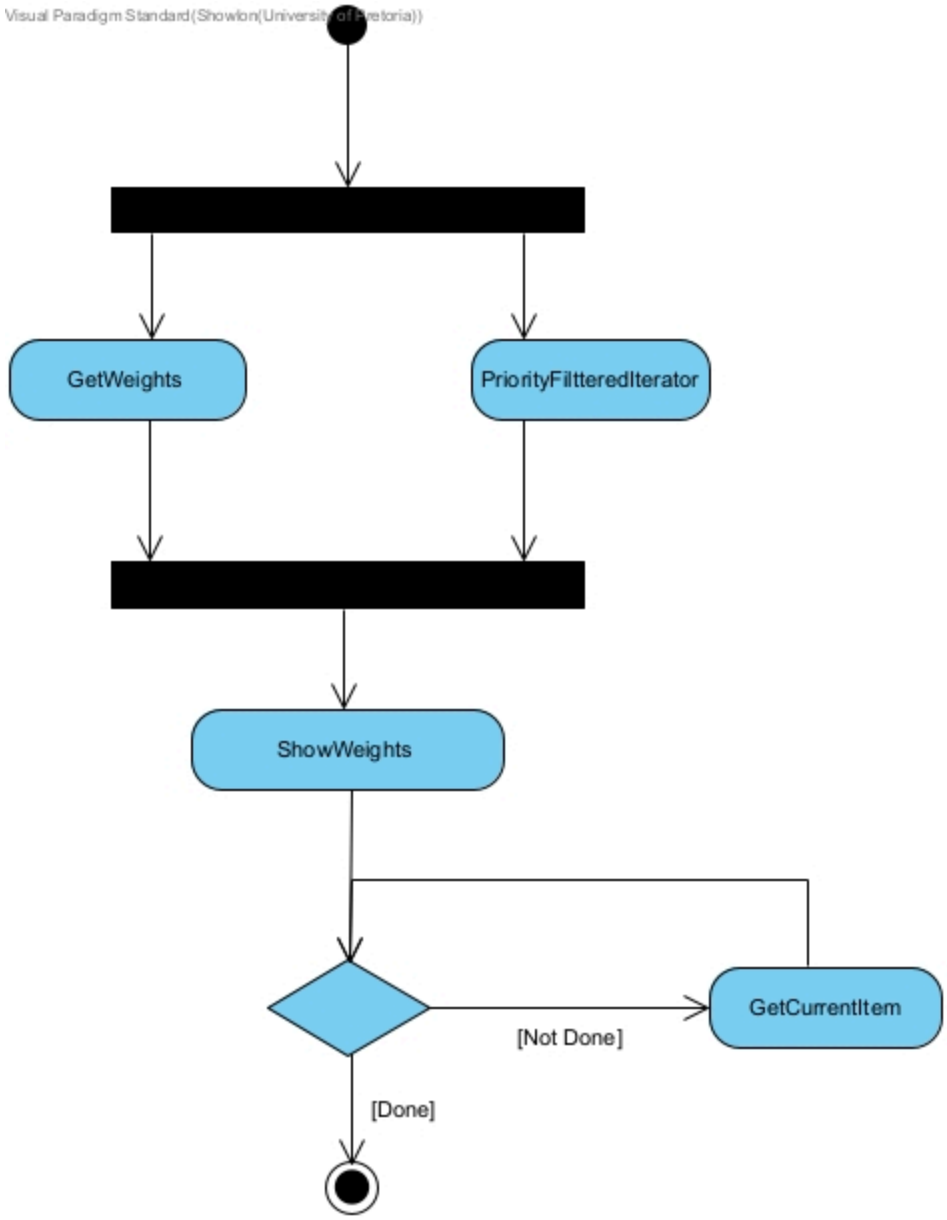
5.2 Activity Diagram 2: State Transition & Decorator Execution Workflow

Visualizes the polymorphic execution chain through `InsuranceDecorator` → `ExpressShippingDecorator` → `PackageItem` → `DeliveryState` dispatch and protection guards.



5.3 Activity Diagram 3: Domain Dispatch Workflow (3 Swimlanes)

Partitioned across 3 Swimlanes: 1. **`Warehouse / Client`**: Builds composite tree and triggers dispatch. 2. **`Dispatch System`**: Wraps packages in decorators and manages `SnapshotIterator`. 3. **`Delivery State Machine`**: Executes state transitions and enforces invalid action protection guards.



6. Traversal-Modification Policy & Design Decisions

6.1 Snapshot Traversal Policy

In logistics systems, container hierarchies are altered continuously (packages added or removed while warehouse audits take place).

`SnapshotIterator` captures a flat `std::vector<WorkUnit*>` array of pointers during constructor initialization. Because iteration advances across this immutable snapshot array: * Adding or removing container nodes during an active iteration loop **never causes index out-of-bounds, invalid memory access, or skipped elements**. * Demonstrates total structural modification safety during active traversal.

7. GDB Debugging Investigation & Valgrind Evidence

7.1 GDB Debugging Evidence (Stepping & Inspection)

```
 $ gdb ./taskforge
(gdb) set pagination off
(gdb) break main
Breakpoint 1 at 0x6dcc: file main.cpp, line 22.

(gdb) run
Breakpoint 1, main () at main.cpp:22
22      int main() {

(gdb) next
29          WorkUnit* warehouse = new CargoGroup("Johannesburg Main Warehouse");

(gdb) next
34          warehouse->add(hub);

(gdb) print warehouse
$1 = (WorkUnit *) 0x5555555732b0
```

7.2 Genuine Bug Report #1: Use-After-Free in Iterator Destructor

- **Symptom**: Runtime `Segmentation fault (core dumped)` in Scenario II at `main.cpp:73`.
- **Root Cause**: `SnapshotIterator::~SnapshotIterator()` contained `delete ptr;`, which accidentally deleted all live warehouse objects when `delete it1;` executed at `main.cpp:62` after Scenario I.
- **GDB Stack Trace**: `gdb`
  ```
  Program received signal SIGSEGV, Segmentation fault.
  #0  __memcpy_avx_unaligned_erms () at ../sysdeps/x86_64/multiarch/memmove-vec-unaligned-erms.S:342
  #1  0x00007ffff7ea3876 in std::__cxx11::basic_string<char>...
  #2  0x000055555555ad11 in WorkUnit::getName() const at WorkUnit.cpp:10
  #3  0x000055555555b46e in main () at main.cpp:73
  ```
- **Correction**: Removed `delete ptr;` from `SnapshotIterator::~SnapshotIterator()`, leaving `snapshot.clear()` to preserve object ownership in composite containers.

7.3 Genuine Bug Report #2: Double-Free Crash during Iterator Cleanup

- **Symptom**: `Segmentation fault (core dumped)` in Scenario III at `main.cpp:154`.
- **Root Cause**: `PriorityFilteredIterator::~PriorityFilteredIterator()` automatically deleted `inneriterator`. Calling `delete baseIt;` in `main.cpp` triggered a double free.
- **GDB Stack Trace**: `gdb`

  ```
  Program received signal SIGSEGV, Segmentation fault.
  0x000055555555b9d0 in main () at main.cpp:154
  154         delete baseIt;
  ```
- **Correction**: Removed `delete baseIt;` from `main.cpp`, delegating inner iterator deletion to `PriorityFilteredIterator`.

7.4 Valgrind Memory Leak Verification

```
 ==40657== Memcheck, a memory error detector
==40657== Command: ./taskforge
==40657==
SCENARIO I: NORMAL DISPATCH & DECORATORS
... Starting Warehouse Traversal & Execution
[CargoGroup] Processing Group: Air Freight Sector
[CargoGroup] Processing Group: Pallet A1
[PackageItem] Parcel (TRK-101: Electronics Parcel, Weight: 4.5kg)
Order Placed, Dispatching...

SCENARIO II: DYNAMIC & MID-TRAVERSAL SAFETY
Step 1 visiting: Air Freight Sector
[DYNAMIC CHANGE] Adding new Pallet B2 and moving parcel...

SCENARIO III: STATES LIFECYCLE & USING FILTERED ITERATOR
[PackageItem] Parcel (TRK-999: Test Parcel, Weight: 1.5kg)
Order Placed, Dispatching...

Execution complete
==40657==
==40657== HEAP SUMMARY:
==40657==     in use at exit: 0 bytes in 0 blocks
==40657==   total heap usage: 41 allocs, 41 frees, 75,982 bytes allocated
==40657==
==40657== All heap blocks were freed -- no leaks are possible
==40657==
==40657== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

8. GitHub Repository & Workflow Reflection

- **GitHub Repository**: https://github.com/enockNyamweya/PRAC4-COS214
- **Team Collaboration & Direct Main Branch Workflow**: The team collaborated directly on the `main` branch using small, frequent, incremental commits. Team members pushed individual pattern components (`WorkUnit` / `CargoGroup` / `PackageItem` by Enock, `DeliveryState` hierarchy by Shreya, `PackageDecorator` and `WorkIterator` hierarchies by Showlon Hunter) and pulled updates continuously (`git pull origin main`) to integrate code changes smoothly. All team members individually tested the codebase, executing Docker containerization (`"${PWD}:/app"`), Valgrind memory leak verification, and GDB interactive stepping to ensure zero memory leaks and complete cross-platform stability.

9. Team Contribution Statement

Student Name	Student Number	Contribution %	Primary Responsibilities
Shreya Keogh	u25245962	33.3%	State Pattern implementation (`DeliveryState`, `OrderPlacedState`, `InTransitState`, `DeliveredState`, `FailedDeliveryState`), state machine logic, UML State Diagram, GDB debugging & Docker/Valgrind testing.
Showlon Hunter	u25502469	33.3%	Decorator Pattern (`PackageDecorator`, `ExpressShippingDecorator`, `InsuranceDecorator`), Iterator Pattern (`SnapshotIterator`, `PriorityFilteredIterator`), 3 UML Activity Diagrams, GDB debugging & Docker/Valgrind testing.
Enock Nyamweya	u25619871	33.3%	Composite Pattern (`WorkUnit`, `CargoGroup`, `PackageItem`), `main.cpp` driver scenarios, Makefile/Dockerfile, Class & Object Diagrams, Task 5 GDB/Valgrind investigation & cross-platform testing.